

TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE ENGINEERING AND AUTOMATICS

REPORT

on laboratory work no. 4

Course: **Operating Systems**

Topic: The xv6 Shell from the Outside

Done by: st. gr. FAF-231
Vitalie Vasilean

Checked by:
univ. assist. / lect. Patricia Reiþman

Chişinău, 2026

Purpose of the work

To study the architecture and behavior of a Unix command interpreter by directly comparing the minimalist shell in xv6 (`user/sh.c`) with the advanced GNU `bash` interpreter in Linux. Understanding the shell's role in lexical analysis of arguments, I/O descriptor redirection, creating pipes, variable evaluation, wildcard expansion (globbing), quote removal, and execution of internal (*built-in*) versus external commands.

1 Part 1: Fix sleep for the current xv6

In the current versions of xv6 (starting August 2025), the `sleep()` system call was renamed to `pause()`. During this stage, the call declaration in `user/user.h` was verified, the source code `user/sleep.c` was updated, and the target's presence in `Makefile` was confirmed.

When running the verification command in the header file:

```
vitalie@LinuxLabs:~/xv6-riscv$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
```

The result confirms that the new call is named `pause`. The `user/sleep.c` utility code was updated accordingly:

```
1 #include "kernel/types.h"
2 #include "user/user.h"
3
4 int
5 main(int argc, char *argv[])
6 {
7     if(argc != 2) {
8         fprintf(2, "usage: sleep <ticks>\n");
9         exit(1);
10    }
11    pause(atoi(argv[1])); // was sleep() before Aug 2025
12    exit(0);
13 }
```

After recompiling the kernel with `make qemu`, the utility was successfully tested in the xv6 shell:

```
$sleep 20$ sleep
usage: sleep <ticks>
```

```
25:int pause(int);
```

Figure 1 – The grep result in `user.h`

[Updated user/sleep.c source code with the pause() function]

Figure 2 – The updated `sleep.c`

[Execution of \$ sleep 20 followed by usage: sleep <ticks> message]

Figure 3 – `sleep` running in xv6, with and without an argument

2 Part 2: bash versus the xv6 shell

A set of 10 comparative commands was executed in both shells: in **bash** (in the `/shtest` directory containing the `README` file) and in the **xv6** interpreter.

2.1 Comparative Table

Command	Tests	bash (Linux)	xv6 shell
<code>echo hello > f; cat < f</code>	redirection	Prints hello . Correctly redirects descriptors 1 and 0.	Prints hello . Redirects files via <code>open</code> , <code>dup</code> .
<code>ls grep READ</code>	pipe	Prints README . Connects stdout to stdin via <code>pipe</code> .	Prints README . Complete implementation of anonymous pipes.
<code>(echo a; echo b) wc</code>	grouping, list	Prints 2 2 4 (lines, words, bytes).	Prints 2 2 4 . Evaluates the grouped subshell.
<code>echo one; echo two</code>	sequence	Prints sequentially one , then two .	Prints sequentially one , then two (the <code>;</code> operator).
<code>echo bg &</code>	background	Prints the background PID [1] 11330 and bg .	Prints bg asynchronously, without blocking the prompt.
<code>ls *</code>	globbing	Expands <code>*</code> to files in the folder (README f).	Error: ls: cannot open *. Looks literally for the file *.
<code>echo \$HOME</code>	variables	Replaces the variable with the path /home/vitalie .	Prints the literal string \$HOME . No variable support.
<code>echo "a b"</code>	quotes	Prints a b . Quotes are removed by the shell.	Prints "a b" quotes included.
<code>mkdir d; cd d; ls ..</code>	PATH	Creates d , enters it and executes <code>ls ..</code> via <code>\$PATH</code> .	<code>ls ..</code> fails (exec ls failed) due to lack of <code>PATH</code> variable.
<code>arrow up, Tab, Ctrl+R</code>	history, completion	Autocompletes the name and navigates history.	Prints raw control characters (no advanced terminal support).

2.2 Observe

1. **Common features vs. bash exclusive:** Both interpreters offer robust support at the system call level for: I/O redirection (`<`, `>`), pipes (`|`), sequential lists (`;`), command grouping, and asynchronous background execution (`&`). Exclusive **bash** features are: wildcard expansion (globbing `*`), environment variable substitution (`$`), quote interpretation, executable searching via `PATH`, command history, and automatic completion.
2. **The behavior of `ls` after `cd d` in **xv6**:** In **xv6**, the `ls` utility stops working inside directory **d** because the interpreter lacks a `PATH` environment variable specifying binary

directories. The shell searches for the executable binary exclusively in the current directory. Since the `ls` binary is in the root directory, the functional command called from `d` is `../ls ...`.

3. **Impact of missing features:** In daily use, the most desired feature is autocompletion via the Tab key and command history, without which productivity drops significantly. From a development complexity perspective, the easiest feature to integrate into `sh.c` would be removing delimiting quotes during parameter processing.

[Terminal bash: commands executed in ~/shtest]

Figure 4 – The commands run in bash

[Terminal xv6: running the same commands at the \$ prompt]

Figure 5 – The same commands run in xv6

[xv6: exec ls failed followed by successful run of ../ls ..]

Figure 6 – `ls` failing after `cd d`, and the working alternative

3 Part 3: Who expands *? A program that prints its argv

The utilities `args.c` in `xv6` and `args_linux.c` in Linux were implemented to analyze how command-line parameters are passed to the `argv[]` array.

Results obtained in Linux (`bash`):

```
vitalie@LinuxLabs:~/shtest$ ./args one two three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]

vitalie@LinuxLabs:~/shtest$ ./args *.c "x y"
argc = 3
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [x y]

vitalie@LinuxLabs:~/shtest$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/vitalie]

vitalie@LinuxLabs:~/shtest$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Results obtained in `xv6`:

```
$ args one two three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]

$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]

$args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
```

3.1 Observe

1. Comparison of the output for `args *.c "x y"`:

- In `bash`: The program received 3 arguments (`argc = 3`). `*.c` was automatically expanded to `args_linux.c`, and `"x y"` was treated as a single argument `[x y]`, with quotes removed.

- In `xv6`: The program received 4 arguments (`argc = 4`). `*.c` was not expanded, arriving literally as `[* .c]`. Because `xv6` does not recognize quotes, the space between `x` and `y` divided the string into two separate arguments: `["x"]` and `[y"]`.
2. **Who expands `*.c` and removes quotes:** The `**command interpreter (shell)**` is the entity responsible for expanding files and stripping quotes. Neither the operating system kernel nor the executable program performs this operation; the executable directly receives the processed arguments from the shell via the `exec` call. In `xv6`, the lack of this logic in the interpreter causes all characters to be passed raw.
 3. **Behavior of multiple spaces:** In both shells, sequences of multiple spaces are interpreted as lexical separators. The shell ignores them and separates words into clean individual arguments.

[args.c code and the \$U/_args\ directive in Makefile]

Figure 7 – `args.c` and the `UPROGS` line

[Execution of args commands in xv6 terminal]

Figure 8 – `args` output in `xv6`

[Execution of ./args commands in Linux bash]

Figure 9 – `args_linux` output in `bash`

4 Part 4: Read the shell

The source file `user/sh.c` (having 499 lines) was analyzed using the `wc` and `grep` utilities:

```
vitalie@LinuxLabs:~/xv6-riscv$ wc -l user/sh.c
499 user/sh.c

vitalie@LinuxLabs:~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC 1
9:#define REDIR 2
10:#define PIPE 3
11:#define LIST 4
12:#define BACK 5
14:#define MAXARGS 10

vitalie@LinuxLabs:~/xv6-riscv$ grep -n "case " user/sh.c
75: case EXEC:
83: case REDIR:
93: case LIST:
101: case PIPE:
125: case BACK:
...
vitalie@LinuxLabs:~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:     fprintf(2, "cannot cd %s\n", cmd + 3);
```

4.1 Observe

1. **The five types of commands and the characters that produce them:**

- EXEC (1) – Simple commands/standard executables, space-delimited.
- REDIR (2) – I/O redirection, produced by `<` and `>`.
- PIPE (3) – Anonymous pipe between processes, produced by `|`.
- LIST (4) – Sequence of independent commands, produced by `;`.
- BACK (5) – Asynchronous background execution, produced by `&`.

2. **Where `cd` is handled and why it is not a separate program:** The `cd` command is handled directly in the main loop of `main()` (line 170 of `user/sh.c`), directly calling the `chdir(cmd + 3)` system call. It cannot be a separate external program because running an external executable requires creating a new process via `fork()`. Changing the working directory in a child process would only affect that child, and the parent process (the shell) would remain in the previous directory upon execution completion.

3. **Searching for `*` and `$` in `sh.c`:** There is no line of code in `sh.c` that checks or processes the `*` or `$` characters. This explains the results from Part 2 and Part 3: the xv6 shell transfers these characters unchanged directly into the programs' parameter list.

```
8:#define EXEC 1 ... 12:#define BACK 5
```

Figure 10 – The command kinds defined in `sh.c`

```
75: case EXEC: ... 125: case BACK:
```

Figure 11 – Where each kind is run

```
170: fprintf(2, "cannot cd %s\n", cmd + 3);
```

Figure 12 – The code that handles cd

Conclusions

In this laboratory work, I explored the architecture of the Unix command interpreter by comparing the minimalist shell in xv6 with GNU `bash` in Linux.

The experiments practically demonstrated the clear separation of responsibilities between the kernel and the command interpreter: while the kernel provides the basic primitives (`fork`, `exec`, `pipe`, `chdir`), the shell is responsible for the lexical parsing of the command line. I experimentally demonstrated that wildcard expansion (`*`), environment variable interpretation (`$`), and quote stripping are tasks performed exclusively by the interpreter before transferring parameters to applications. Furthermore, analyzing the source code in `sh.c` explained the mechanism by which commands like `cd` must be native (*built-in*), illustrating the fundamental principles of process management in Unix operating systems.